Performance Optimization Strategy

set a target, measure, fix the critical path, verify

Watch the 4-min video

In one line: to make something faster, set a measurable target, trace a real request to see where time goes, optimise the biggest part of the critical path first (Amdahl's Law caps the gain from small parts), favour the usual big wins (fewer round trips, parallel independent calls, caching, precomputation, indexes, less data), then verify under realistic load and keep the measurement.

1 "The dashboard is slow, make it faster"

Where do you start? Rewrite it in a faster language? Add a cache everywhere? Without a strategy you'll optimise the wrong thing. The answer is almost never to start rewriting code.

2 Measure first

Define the **target**: "page load under 500 ms at p95". Then **trace** a real request end to end and see where the time goes: distributed tracing (OpenTelemetry) shows each span, the database queries, downstream calls and rendering. The answer is often surprising. Profiling (lesson 43) then shows what's hot inside each service.

3 The critical path and Amdahl's Law

Optimise the **critical path**, the longest chain of dependent work. Speeding up anything off that path changes nothing. **Amdahl's Law** sets the limit: if a part takes 20% of the time, making it infinitely fast saves only 20%. Optimising a step that's 5% of the request is wasted effort; go after the biggest span first.



make part X infinitely fast → the request is at most 20% faster

speedup ≤ 1 / (1 − share) = 1 / 0.8 = 1.25× · so go after the biggest span first

Figure 1. Amdahl's Law: the share a part takes is the most you can gain by speeding it up.

4 The usual big wins: round trips

- Replace **N+1 queries** with one query or a join (Phase 2, lesson 60).
- **Batch** calls together (lesson 39).
- Run **independent calls in parallel**: with asyncio, `gather` lets three 200 ms calls finish in 200 ms, not 600.

Round trips add up fast when each one costs milliseconds.

5 The rest of the toolbox

- **Cache** expensive results (lessons 15 to 33).
- **Precompute** aggregates ahead of time instead of on every request.
- **Paginate**, and return only the fields you need.
- Add the right **index**.
- Use the right **data structure**: a set, not a list, for membership checks.
- Move non-urgent work to **background jobs** (lesson 37).

Each of these is cheaper than rewriting the service.

WORKED EXAMPLE: A 2-SECOND PRODUCT PAGE

Tracing shows four downstream calls made one after another: user, product, recommendations and reviews, around 400 ms each. They're independent, so call them in parallel with `asyncio.gather`. The page now waits only for the slowest call. Add a cache for recommendations, and the page loads in about **400 ms**.

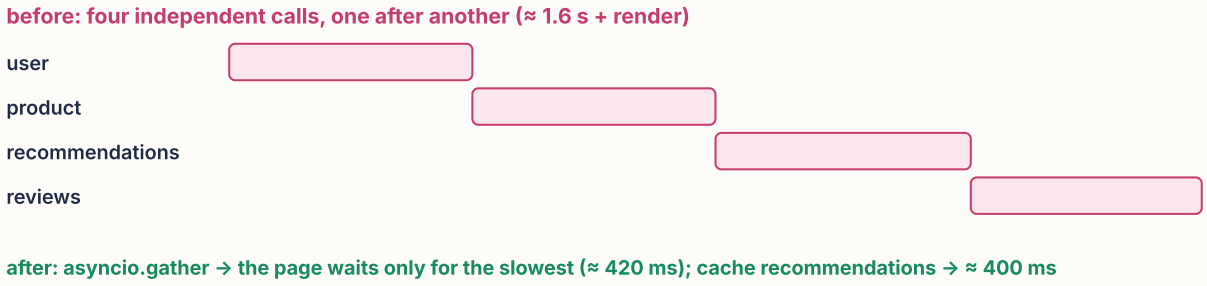


Figure 2. The trace that explains the slow page: four sequential spans that don't depend on each other.

```
# before: ~1,600 ms
user = await get_user(uid); product = await get_product(pid)
recs = await get_recs(uid); reviews = await get_reviews(pid)

# after: ~420 ms (the slowest call)
user, product, recs, reviews = await asyncio.gather(
    get_user(uid), get_product(pid), get_recs(uid), get_reviews(pid))
```

Figure 3. The fix in code: one gather instead of four awaits in a row.

6 Verify, and keep it fixed

Measure again after every change, under realistic load (lesson 44), and compare **percentiles**, not single runs. Stop when you hit the target, because each further gain costs more. Keep the measurement in place, with dashboards and alerts, so a regression doesn't return silently, and write down what changed and why.

COMMON MISTAKES

- Optimising without a target or a baseline.
- Micro-optimising code that's 2% of the request.
- Caching everything before finding the real bottleneck.
- Declaring victory from one fast run on a laptop.

7 Key terms

Critical path: the longest chain of dependent work

Span: one timed operation in a trace

asyncio.gather: run awaitables concurrently, wait for all

Amdahl's Law: speedup limited by the share you improve

Distributed tracing: following one request across services

Precomputation: compute ahead of time, read cheaply

8 Interview questions

Q1. How do you approach a 'make it faster' task? BASIC

Set a measurable target, measure with tracing and profiling, optimise the biggest part of the critical path first, then verify the improvement under realistic load.

Q2. What are the usual big wins? BASIC

Fewer round trips, parallel independent calls, caching and precomputation, indexes, and returning less data.

Q3. A page makes 4 sequential 400 ms calls that don't depend on each other. Fix? SCENARIO

Run them concurrently (asyncio.gather or a thread pool): latency becomes the slowest call, about 400 ms instead of 1.6 s; then cache the slowest if possible.

Q4. Database queries are 10% of request time. Is optimising them worth it? SCENARIO

At most a 10% gain by Amdahl's Law; look first at the span that dominates the critical path.

Q5. Why can parallelising calls make things worse? TRICKY

It multiplies concurrent load on downstream services and the database; if they're near saturation, latency rises. Cap concurrency and watch their utilization.

Q6. When do you stop optimising? TRICKY

When the target is met with headroom; beyond that each gain costs more effort and complexity. Keep monitoring so regressions are caught.

Next: SD-J-015 · Backpressure backendroadmap.com